

React Tutorial

A Complete Guide

14 chapters compiled into one PDF

Snehal Kadiya

+91 9974031480

snehaliteng@gmail.com

<https://snehaliteng.github.io/>

Copyright & Disclaimer

© 2026 Snehal Kadiya. All rights reserved.

This tutorial is intended for personal learning and educational purposes only. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the author.

Please do not print this document unnecessarily. Think of the environment — save paper and trees. Share the digital link instead:

<https://snehaliteng.github.io/tutorials/react/>

Getting Started with React

What is React?

React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It was created by Jordan Walke, a software engineer at Facebook, and first deployed on Facebook's News Feed in 2011 and on Instagram in 2012. React made its source code available at JSConf US in May 2013.

React allows developers to build large, complex web applications that can change data without reloading the page. The main purpose of React is to be fast, scalable, and simple. It works only on the user interface in the application, which corresponds to the view in the MVC template.

Single Page Applications (SPA)

A Single Page Application (SPA) is a web application that loads a single HTML page and dynamically updates that page as the user interacts with the app. React is particularly well-suited for building SPAs because of its component-based architecture and efficient virtual DOM diffing algorithm.

Key Concept: In a traditional multi-page app, every navigation triggers a full page reload. In an SPA built with React, only the necessary parts of the page are re-rendered, resulting in a smoother, faster user experience.

The Virtual DOM

React maintains a lightweight representation of the real DOM in memory, known as the virtual DOM. When the state of an object changes, React updates the virtual DOM first, then compares it with the previous snapshot using a

diffing algorithm. Finally, it calculates the most efficient way to update the real DOM with the minimum number of operations.

```
// Virtual DOM in action
const element = <h1>Hello, World!</h1>;
// React creates a virtual DOM tree
// When state changes, React diffs and patches
```

Learning Roadmap

Here is a recommended path for learning React:

1. **JavaScript Fundamentals:** ES6+ features (arrow functions, destructuring, spread operator, template literals, modules)
2. **React Basics:** JSX, components, props, and state
3. **Intermediate:** Event handling, conditional rendering, lists and keys
4. **Advanced:** Hooks, context API, refs, and custom hooks
5. **Ecosystem:** React Router, state management, testing, and deployment

Installation & Setup

Before you start building React applications, you need to install Node.js and npm. Node.js provides a JavaScript runtime environment, while npm is the package manager used to install React and other dependencies.

Using Create React App

```
npx create-react-app my-app
cd my-app
npm start
```

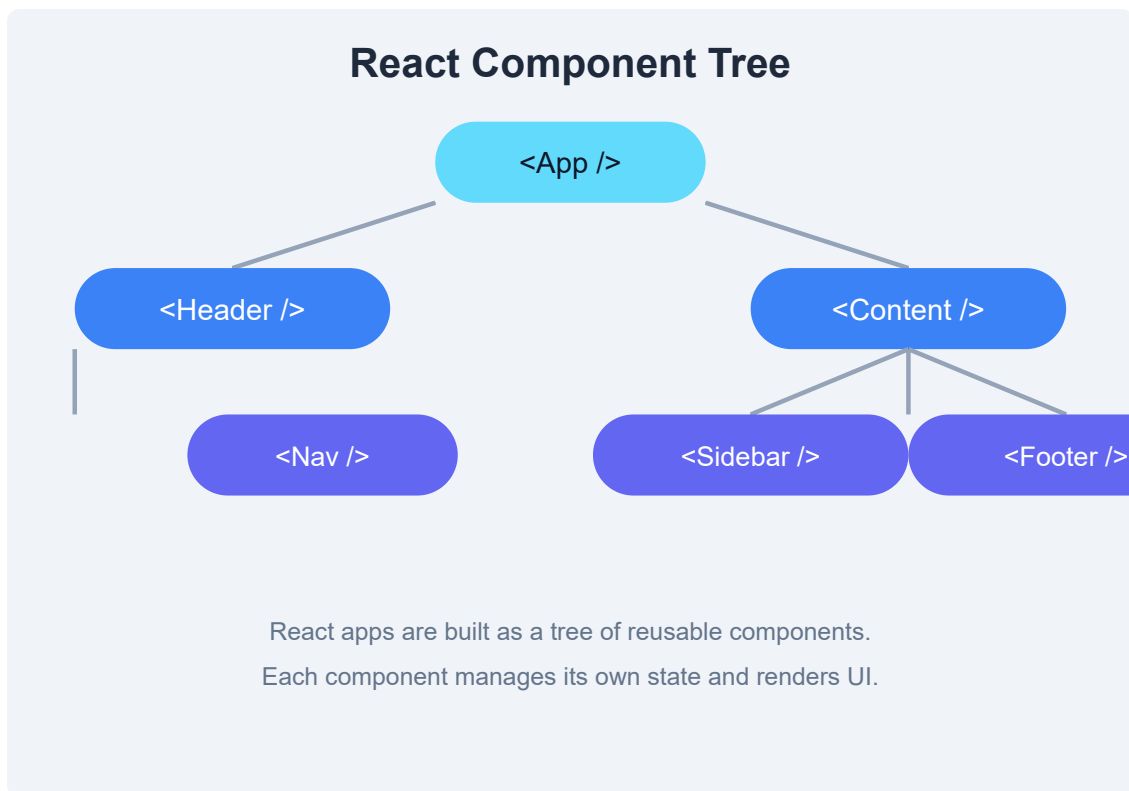
Using Vite (Recommended)

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

Note: As of 2025, Vite has become the recommended tool for new React projects due to its faster build times and smaller bundle sizes compared to Create React App.

Key Features of React

- **Declarative:** React makes it easy to create interactive UIs by declaring how your UI should look based on different states.
- **Component-Based:** Build encapsulated components that manage their own state, then compose them to make complex UIs.
- **Learn Once, Write Anywhere:** React can be used to build web apps (React DOM), mobile apps (React Native), and even desktop apps (Electron + React).
- **Unidirectional Data Flow:** Data flows from parent to child components via props, making the application easier to understand and debug.
- **Rich Ecosystem:** React has a vast ecosystem of libraries, tools, and community support.



Pros & Cons

Advantages

- Fast rendering with virtual DOM
- Reusable component architecture
- Strong community and ecosystem
- SEO-friendly with server-side rendering (Next.js)
- Excellent developer tools (React DevTools)

Disadvantages

- Steep learning curve for beginners
- JSX can be confusing at first
- Frequent updates and breaking changes
- Boilerplate code for complex setups

Folder Structure

A typical React project created with Vite has the following structure:

```
my-app/  
  node_modules/      # Dependencies  
  public/             # Static assets  
  src/  
    App.jsx           # Main application component  
    App.css           # Styles for App component  
    index.css         # Global styles  
    main.jsx          # Entry point  
  index.html          # HTML template  
  package.json        # Project metadata and dependencies  
  vite.config.js      # Vite configuration
```

Practice Task

Create a new React project using Vite. Inside the `App.jsx` file, modify the default content to display your name and a short description of why you want to learn React. Run the development server and verify it works in your browser.

JSX & Components

What is JSX?

JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows you to write HTML-like code directly inside your JavaScript files. JSX makes React code more readable and expressive by combining the markup with the logic that renders it.

```
// JSX syntax example
const element = <h1 className="greeting">Hello, React!</h1>;
```

JSX Rules

- **Single Root Element:** JSX expressions must have one parent element. Use a `<div>` or a React Fragment (`<></>`) to wrap multiple elements.
- **Close All Tags:** Every tag must be closed, including self-closing ones like `<img />` and `<br />`.
- **className Instead of class:** Since `class` is a reserved word in JavaScript, React uses `className` for CSS classes.
- **CamelCase Properties:** HTML attributes like `onclick` become `onClick`, `tabindex` becomes `tabIndex`.

```
// Correct JSX with multiple elements
function Welcome() {
  return (
    <>
      <h1>Hello!</h1>
      <p>Welcome to React.</p>
    </>
  );
}
```

JavaScript Expressions in JSX

You can embed any JavaScript expression inside JSX by wrapping it in curly braces `{ }`. This includes variables, function calls, and mathematical operations.

```
const name = "Alice";
const age = 25;
const element = (
  <div>
    <p>Name: {name}</p>
    <p>Age: {age}</p>
    <p>Next year: {age + 1}</p>
  </div>
);
```

Functional Components

In modern React, components are written as JavaScript functions. A functional component is a function that accepts a `props` object as an argument and returns JSX. Functional components are simpler, easier to test, and with hooks they can manage state and side effects just like class components.

```
// A simple functional component
function Greeting(props) {
  return <h1>Hello, {props.name}!</h1>;
}

// Arrow function version
const Greeting = (props) => {
  return <h1>Hello, {props.name}!</h1>;
};
```

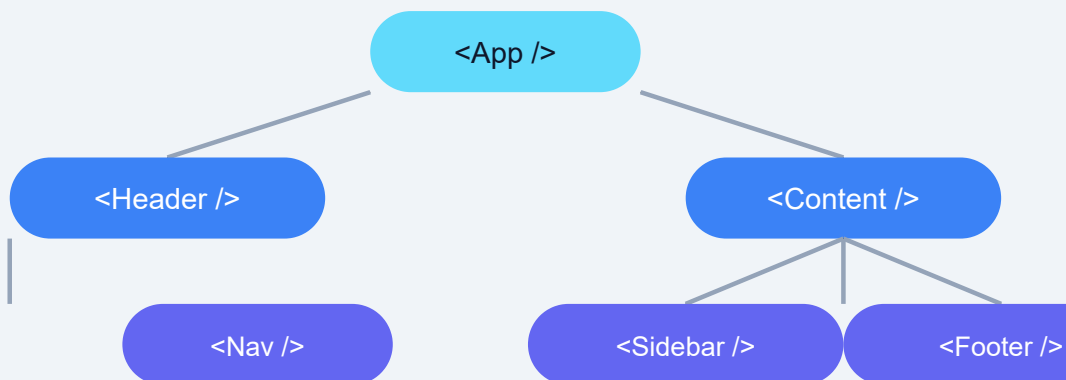
Component Composition

One of React's most powerful features is the ability to compose components by nesting them inside each other. This allows you to build complex interfaces

from small, reusable pieces.

```
function Avatar(props) {  
  return <img src={props.src} alt={props.alt} />;  
}  
  
function UserInfo(props) {  
  return (  
    <div className="user-info">  
      <Avatar src={props.user.avatar} alt={props.user.name} />  
      <h3>{props.user.name}</h3>  
    </div>  
  );  
}  
  
function Comment(props) {  
  return (  
    <div className="comment">  
      <UserInfo user={props.author} />  
      <p>{props.text}</p>  
    </div>  
  );  
}
```

React Component Tree



React apps are built as a tree of reusable components.
Each component manages its own state and renders UI.

Nested Components

Components can be nested to any depth. The parent component passes data down to its children via props. This creates a clear, predictable data flow that makes your application easier to reason about.

Best Practice: Keep components small and focused on a single responsibility. If a component grows too large, split it into smaller sub-components. This improves reusability and testability.

Props

Props (short for properties) are read-only inputs to a React component. They allow data to flow from parent to child components. Props can be strings, numbers, booleans, objects, arrays, functions, or even other components.

```
// Passing props to a component
const element = <Product title="Laptop" price={999} inStock={true} />;

// Receiving props in the component
function Product(props) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>Price: ${props.price}</p>
      {props.inStock ? <p>In Stock</p> : <p>Out of Stock</p>}
    </div>
  );
}
```

Default Exports

React components are typically exported using default exports, allowing them to be imported with any name in other files. This is the standard pattern for single-component files.

```
// In Greeting.jsx
export default function Greeting() {
  return <h1>Hello!</h1>;
}

// In App.jsx
import Greeting from './Greeting';
```

Styling Options

Note: React offers multiple ways to style components. Choose the approach that best fits your project's needs.

- **Inline Styles:** Use a JavaScript object with camelCase properties.
- **CSS Modules:** Scoped CSS files that generate unique class names to avoid collisions.
- **Styled-Components:** A CSS-in-JS library that allows you to write actual CSS in JavaScript.
- **Tailwind CSS:** A utility-first CSS framework that works well with React.

```
// Inline styles
const style = { color: 'blue', fontSize: '18px' };
return <p style={style}>Styled text</p>;

// Styled-components
import styled from 'styled-components';
const Button = styled.button`
  background: #3b82f6;
  color: white;
  padding: 8px 16px;
  border: none;
  border-radius: 4px;
`;
```

Practice Task

Create a `ProductCard` component that accepts `name`, `price`, and `image` as props. Then create an `App` component that renders three `ProductCard` components with different product data. Use inline styles to give each card a border, padding, and margin.

Props & State

Understanding Props

Props (properties) are the mechanism by which data flows from a parent component to a child component in React. They are read-only, meaning a child component cannot modify the props it receives. This unidirectional data flow ensures that the application is predictable and easier to debug.

```
// Parent passing props
function App() {
  return <User name="John" age={30} isActive={true} />;
}

// Child receiving props
function User(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      {props.isActive && <span>Active</span>}
    </div>
  );
}
```

The Children Prop

The `children` prop is a special prop that allows you to pass JSX content between the opening and closing tags of a component. This is useful for creating wrapper or layout components.

```
function Card(props) {
  return (
    <div className="card">
      {props.children}
    </div>
  );
}

// Usage
<Card>
  <h2>Card Title</h2>
  <p>This content is passed as children.</p>
</Card>
```

Prop-Types Validation

React provides a built-in type-checking feature called `PropTypes` to validate the props that a component receives. This helps catch bugs during development by warning you when props of the wrong type are passed.

```
import PropTypes from 'prop-types';

function Product({ name, price, category }) {
  return (
    <div>
      <h3>{name}</h3>
      <p>${price} - {category}</p>
    </div>
  );
}

Product.propTypes = {
  name: PropTypes.string.isRequired,
  price: PropTypes.number.isRequired,
  category: PropTypes.string,
};

Product.defaultProps = {
  category: 'General',
};
```

State in React

State represents the data that changes over time in a component. Unlike props, state is managed internally by the component and can be updated using setter functions. When state changes, React re-renders the component to reflect the new values.

State in Class Components (Legacy)

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>+</button>
      </div>
    );
  }
}
```

State in Functional Components (Modern)

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

Note: Functional components with hooks are now the standard way to write React components. Class components are still supported but are considered legacy.

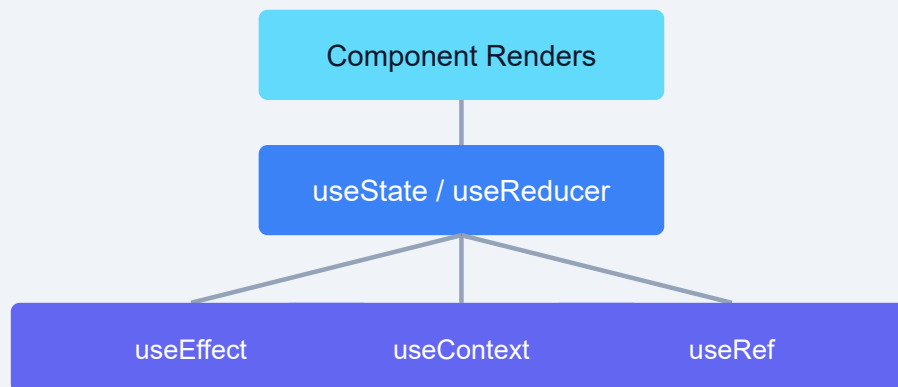
Stateless vs Stateful Components

A **stateless component** (also called a presentational or dumb component) does not manage any state of its own. It receives data via props and renders it. A **stateful component** (also called a smart or container component) manages state and often passes it down to stateless children.

```
// Stateless (presentational)
function UserDisplay({ name, email }) {
  return <div>{name} {email}</div>;
}

// Stateful (container)
function UserContainer() {
  const [user, setUser] = useState(null);
  // fetch user data...
  return <UserDisplay name={user.name} email={user.email} />;
}
```

React Hooks Data Flow



Hooks let you use state and lifecycle features in functional components.

Lifecycle with Hooks

The `useEffect` hook combines the functionality of three class component lifecycle methods: `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`.

```
import { useState, useEffect } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  // componentDidMount + componentDidUpdate
  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(s => s + 1);
    }, 1000);

    // componentWillUnmount (cleanup)
    return () => clearInterval(interval);
  }, []); // Empty dependency array = run once on mount

  return <p>Timer: {seconds}s</p>;
}
```

Key Insight: The dependency array in `useEffect` controls when the effect runs. An empty array means it runs only on mount. Omitting the array means it runs after every render. Including specific variables means it runs only when those variables change.

Lifting State Up

When multiple components need to share the same state, it is a best practice to lift the state up to their closest common ancestor. This parent component then passes the state and event handlers down as props.

```
function TemperatureCalculator() {
  const [temperature, setTemperature] = useState('');

  return (
    <div>
      <Input temp={temperature} onTempChange={setTemperature} />
      <Display temp={temperature} scale="c" />
      <Display temp={temperature} scale="f" />
    </div>
  );
}
```

Practice Task

Build a simple `TodoList` application. Create a parent component that holds an array of todo items in state. Render each todo item using a stateless `TodoItem` component. Add an input field and a button to add new todos to the list. Use prop-types to validate the props of `TodoItem`.

Handling Events in React

Event Handling in React

Event handling in React is similar to handling events in the DOM, but with a few key differences. React events are named using camelCase (`onClick` instead of `onclick`), and you pass a function as the event handler rather than a string. React uses its own synthetic event system to ensure cross-browser compatibility.

```
// Basic event handling
function ClickButton() {
  const handleClick = () => {
    alert('Button clicked!');
  };

  return <button onClick={handleClick}>Click Me</button>;
}
```

Common Event Types

- **onClick:** Triggered when an element is clicked.
- **onChange:** Triggered when the value of an input, select, or textarea changes.
- **onSubmit:** Triggered when a form is submitted.
- **onMouseEnter / onMouseLeave:** Triggered when the mouse enters or leaves an element.
- **onFocus / onBlur:** Triggered when an element gains or loses focus.
- **onKeyDown / onKeyUp:** Triggered when a key is pressed or released.

```
// Multiple event handlers
function InputExample() {
  const [value, setValue] = useState('');

  const handleChange = (e) => setValue(e.target.value);
  const handleFocus = () => console.log('Focused');
  const handleKeyDown = (e) => {
    if (e.key === 'Enter') {
      alert(`Submitted: ${value}`);
    }
  };

  return (
    <input
      type="text"
      value={value}
      onChange={handleChange}
      onFocus={handleFocus}
      onKeyDown={handleKeyDown}
    />
  );
}
```

Synthetic Events

React wraps native browser events into a `SyntheticEvent` object that provides a consistent interface across all browsers. The `SyntheticEvent` has the same properties as native events, including `bubbles`, `cancelable`, `currentTarget`, `defaultPrevented`, `eventPhase`, `isTrusted`, `nativeEvent`, `preventDefault()`, `stopPropagation()`, `target`, `timestamp`, and `type`.

Performance Note: React's synthetic events are pooled for performance. This means the event object is reused and its properties are nullified after the event handler has finished executing. If you need to access an event property asynchronously (e.g., inside a `setTimeout`), you must call `e.persist()` to remove the event from the pool.

```
// Synthetic event object
function Form() {
  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    console.log('Form submitted');
    console.log('Target:', e.target);
    console.log('Type:', e.type);
  };

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Passing Arguments to Event Handlers

When you need to pass additional arguments to an event handler, you can use an arrow function or the `bind` method. Arrow functions are the more common and readable approach in modern React.

```
// Using arrow function (recommended)
function ItemList({ items }) {
  const handleClick = (id, name) => {
    console.log(`Clicked item ${id}: ${name}`);
  };

  return (
    <ul>
      {items.map(item => (
        <li key={item.id} onClick={() => handleClick(item.id, item.name)}>
          {item.name}
        </li>
      ))}
    </ul>
  );
}
```

Practical Examples

Click Counter

```
function ClickCounter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Clicks: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Toggle Button

```
function ToggleButton() {
  const [isOn, setIsOn] = useState(false);

  return (
    <button
      onClick={() => setIsOn(prev => !prev)}
      style={{ background: isOn ? '#22c55e' : '#ef4444' }}
    >
      {isOn ? 'ON' : 'OFF'}
    </button>
  );
}
```

Input Change Handler

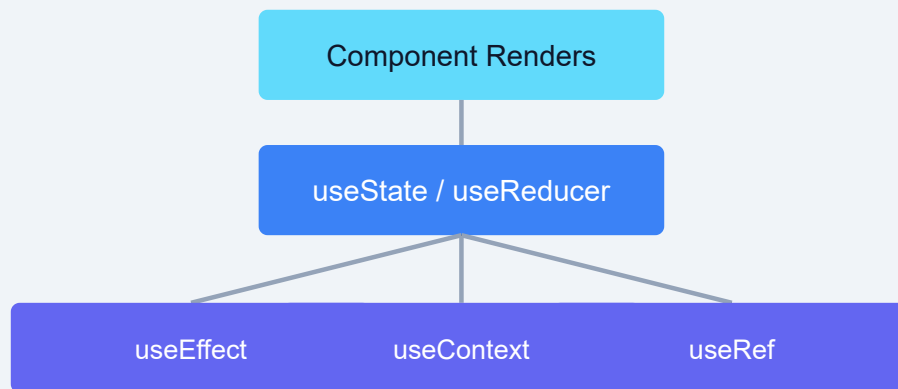
```
function SignupForm() {
  const [formData, setFormData] = useState({
    username: '',
    email: '',
    password: '',
  });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Form data:', formData);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="username" value={formData.username} onChange={handleChange} />
      <input name="email" type="email" value={formData.email} onChange={handleChange} />
      <input name="password" type="password" value={formData.password} onChange={handleChange} />
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

React Hooks Data Flow



Hooks let you use state and lifecycle features in functional components.

Event-Aware Components Pattern

A common pattern in React is to create components that accept event handlers as props. This allows the parent component to control what happens when events occur, making the child component reusable in different contexts.

```
function ActionButton({ label, onAction, disabled }) {  
  return (  
    <button onClick={onAction} disabled={disabled}>  
      {label}  
    </button>  
  );  
}  
  
// Usage  
<ActionButton label="Save" onAction={handleSave} disabled={isSaving} />  
<ActionButton label="Delete" onAction={handleDelete} disabled={false} />
```

Practice Task

Create a `ColorPicker` component that displays three buttons (Red, Green, Blue). When a button is clicked, the background color of a preview box changes to the corresponding color. Use the `onClick` event handler and pass arguments to identify which color was clicked. Also add a reset button that changes the background back to white.

React Hooks

Introduction to Hooks

React Hooks were introduced in React 16.8 (2019) as a way to use state and other React features without writing a class. Before hooks, functional components were limited to rendering props and could not manage state or side effects. Hooks solve this by providing dedicated functions for state management, side effects, context consumption, and more.

Why Hooks? Hooks allow you to reuse stateful logic without changing your component hierarchy. They let you split a complex component into smaller functions based on related pieces of logic, rather than forcing you to split by lifecycle methods.

useState

The `useState` hook lets you add state to functional components. It returns an array with two elements: the current state value and a setter function to update it. You can call `useState` multiple times in a single component to manage different pieces of state.

```
// Counter with useState
function Counter() {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + step)}>+{step}</button>
      <button onClick={() => setStep(s => s + 1)}>Increase Step</button>
    </div>
  );
}
```

Form Fields with useState

```
function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});

  const handleSubmit = (e) => {
    e.preventDefault();
    const newErrors = {};
    if (!email) newErrors.email = 'Email is required';
    if (!password) newErrors.password = 'Password is required';
    setErrors(newErrors);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      {errors.email && <p>{errors.email}</p>}
      <input type="password" value={password} onChange={e => setPassword(e.target.value)} />
      {errors.password && <p>{errors.password}</p>}
      <button type="submit">Login</button>
    </form>
  );
}
```

useEffect

The `useEffect` hook lets you perform side effects in functional components. Side effects include data fetching, subscriptions, timers, and manually changing the DOM. The effect runs after every render by default, but you can control this with the dependency array.

```
// Data fetching with useEffect
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let cancelled = false;

    async function fetchUser() {
      setLoading(true);
      const response = await fetch(`/api/users/${userId}`);
      const data = await response.json();
      if (!cancelled) {
        setUser(data);
        setLoading(false);
      }
    }

    fetchUser();

    return () => {
      cancelled = true; // Cleanup on unmount or userId change
    };
  }, [userId]);

  if (loading) return <p>Loading...</p>;
  return <h2>{user.name}</h2>;
}
```

useContext

The `useContext` hook lets you subscribe to React context without introducing nesting. It accepts a context object (created with `React.createContext`) and

returns the current context value. This is useful for global state like themes, locale settings, or authentication status.

```
const ThemeContext = React.createContext('light');

function ThemedButton() {
  const theme = useContext(ThemeContext);
  return (
    <button style={{
      background: theme === 'dark' ? '#333' : '#fff',
      color: theme === 'dark' ? '#fff' : '#333'
    }}>
      {theme} mode
    </button>
  );
}

function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={theme}>
      <ThemedButton />
      <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
        Toggle Theme
      </button>
    </ThemeContext.Provider>
  );
}
```

useRef

The `useRef` hook returns a mutable ref object whose `.current` property persists across re-renders. It is commonly used to access DOM elements directly or to store mutable values that should not trigger re-renders when changed.

```
// DOM ref example
function AutoFocusInput() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus();
  }, []);

  return <input ref={inputRef} type="text" />;
}

// Storing a mutable value (no re-render)
function Timer() {
  const intervalRef = useRef();

  const startTimer = () => {
    intervalRef.current = setInterval(() => {
      console.log('Tick');
    }, 1000);
  };

  const stopTimer = () => {
    clearInterval(intervalRef.current);
  };

  return (
    <div>
      <button onClick={startTimer}>Start</button>
      <button onClick={stopTimer}>Stop</button>
    </div>
  );
}
```

useReducer

The `useReducer` hook is an alternative to `useState` for managing complex state logic. It accepts a reducer function and an initial state, returning the current state and a dispatch function. It is particularly useful when state transitions depend on previous state or when multiple sub-values are involved.

```

function todoReducer(state, action) {
  switch (action.type) {
    case 'ADD':
      return [...state, { id: Date.now(), text: action.payload, done: false }];
    case 'TOGGLE':
      return state.map(t =>
        t.id === action.payload ? { ...t, done: !t.done } : t
      );
    case 'DELETE':
      return state.filter(t => t.id !== action.payload);
    default:
      return state;
  }
}

function TodoApp() {
  const [todos, dispatch] = useReducer(todoReducer, []);
  const [input, setInput] = useState('');

  const addTodo = () => {
    if (input.trim()) {
      dispatch({ type: 'ADD', payload: input });
      setInput('');
    }
  };

  return (
    <div>
      <input value={input} onChange={e => setInput(e.target.value)} />
      <button onClick={addTodo}>Add</button>
      {todos.map(t => (
        <div key={t.id}>
          <span style={{ textDecoration: t.done ? 'line-through' : 'none' }}
            onClick={() => dispatch({ type: 'TOGGLE', payload: t.id })}>
            {t.text}
          </span>
          <button onClick={() => dispatch({ type: 'DELETE', payload: t.id })}>
            </div>
        ))}
    </div>
  );
}

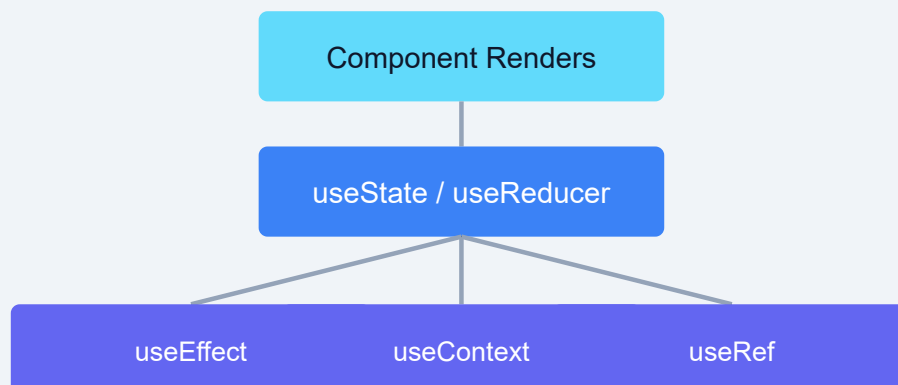
```

useCallback & useMemo

`useCallback` returns a memoized callback function that only changes when one of its dependencies changes. `useMemo` returns a memoized value that is only recomputed when dependencies change. Both are optimization hooks to prevent unnecessary re-renders.

```
function ExpensiveList({ items, filter }) {  
  const filteredList = useMemo(() => {  
    return items.filter(item => item.category === filter);  
  }, [items, filter]);  
  
  const handleClick = useCallback((id) => {  
    console.log('Clicked:', id);  
  }, []);  
  
  return (  
    <ul>  
      {filteredList.map(item => (  
        <li key={item.id} onClick={() => handleClick(item.id)}>{item.name}  
      ))}  
    </ul>  
  );  
}
```

React Hooks Data Flow



Hooks let you use state and lifecycle features in functional components.

Custom Hooks

Custom hooks let you extract component logic into reusable functions. A custom hook is a JavaScript function whose name starts with `use` and that may call other hooks. They enable logic sharing without changing the component hierarchy.

```
// useFetch custom hook
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    fetch(url)
      .then(res => res.json())
      .then(setData)
      .catch(setError)
      .finally(() => setLoading(false));
  }, [url]);

  return { data, loading, error };
}

// Usage
function UsersPage() {
  const { data, loading, error } = useFetch('/api/users');
  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;
  return <p>Users: {data.length}</p>;
}
```

useLocalStorage

```
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      return initialValue;
    }
  });

  const setValue = (value) => {
    setStoredValue(value);
    window.localStorage.setItem(key, JSON.stringify(value));
  };

  return [storedValue, setValue];
}
```

useWindowSize

```
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight,
  });

  useEffect(() => {
    const handleResize = () => {
      setSize({ width: window.innerWidth, height: window.innerHeight });
    };
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return size;
}
```

Practice Task

Create a custom hook called `useDebounce` that takes a value and a delay (in milliseconds) and returns a debounced version of the value. Use this hook in a search input component that fetches search results from a mock API only after the user has stopped typing for 500ms. Display the results in a list below the input.

Layout & UI Components

Layout Components Pattern

A layout component is a reusable component that provides the structural skeleton of your application. It handles common UI elements like headers, sidebars, and footers, while rendering the page-specific content via the `children` prop. This pattern helps maintain a consistent look and feel across different pages.

```
function Layout({ children, sidebar }) {
  return (
    <div className="app-layout">
      <header className="app-header">
        <h1>My App</h1>
      </header>
      <div className="app-body">
        <aside className="app-sidebar">{sidebar}</aside>
        <main className="app-content">{children}</main>
      </div>
      <footer className="app-footer">
        © 2025 My App
      </footer>
    </div>
  );
}

// Usage
function HomePage() {
  return (
    <Layout sidebar={<NavMenu />}>
      <Dashboard />
    </Layout>
  );
}
```

Pagination Component

A pagination component is a common UI pattern for displaying large sets of data across multiple pages. It typically includes previous and next buttons along with page number indicators.

```
function Pagination({ currentPage, totalPages, onPageChange }) {
  const pages = Array.from({ length: totalPages }, (_, i) => i + 1);

  return (
    <div className="pagination">
      <button disabled={currentPage === 1} onClick={() => onPageChange(currentPage - 1)}>
        Previous
      </button>
      {pages.map(page => (
        <button key={page} onClick={() => onPageChange(page)}
          className={page === currentPage ? 'active' : ''}>
            {page}
          </button>
        ))}
      <button disabled={currentPage === totalPages} onClick={() => onPageChange(currentPage + 1)}>
        Next
      </button>
    </div>
  );
}
```

Material UI

Material UI (MUI) is one of the most popular React UI frameworks. It provides a comprehensive set of components that implement Google's Material Design guidelines. MUI components are production-ready, customizable, and accessible.

```

import { Button, Grid, AppBar, Toolbar, Typography, Card } from '@mui/material'

function Dashboard() {
  return (
    <div>
      <AppBar position="static">
        <Toolbar>
          <Typography variant="h6">Dashboard</Typography>
        </Toolbar>
      </AppBar>
      <Grid container spacing={3} sx={{ p: 3 }}>
        <Grid item xs={12} sm={6} md={4}>
          <Card sx={{ p: 2 }}>
            <Typography variant="h5">Card 1</Typography>
            <Button variant="contained">Action</Button>
          </Card>
        </Grid>
        <Grid item xs={12} sm={6} md={4}>
          <Card sx={{ p: 2 }}>
            <Typography variant="h5">Card 2</Typography>
            <Button variant="outlined">Action</Button>
          </Card>
        </Grid>
      </Grid>
    </div>
  );
}

```

React Bootstrap

React Bootstrap is a complete re-implementation of Bootstrap components for React. Unlike the original Bootstrap, React Bootstrap does not rely on jQuery, making it fully compatible with React's declarative approach.

```

import { Container, Row, Col, Card, Button, Navbar, Nav } from 'react-bootstrap'

function LandingPage() {
  return (
    <div>
      <Navbar bg="dark" variant="dark">
        <Container>
          <Navbar.Brand>MySite</Navbar.Brand>
          <Nav>
            <Nav.Link href="#home">Home</Nav.Link>
            <Nav.Link href="#features">Features</Nav.Link>
          </Nav>
        </Container>
      </Navbar>
      <Container className="mt-4">
        <Row>
          {[1, 2, 3].map(i => (
            <Col md={4} key={i}>
              <Card>
                <Card.Body>
                  <Card.Title>Feature {i}</Card.Title>
                  <Card.Text>Description of feature {i}.</Card.Text>
                  <Button variant="primary">Learn More</Button>
                </Card.Body>
              </Card>
            </Col>
          ))}
        </Row>
      </Container>
    </div>
  );
}

```

Tables with Sorting

Data tables are a common UI component in dashboard applications. A sortable table allows users to click column headers to sort the data in ascending or descending order.

```

function SortableTable({ columns, data }) {
  const [sortKey, setSortKey] = useState(null);
  const [sortOrder, setSortOrder] = useState('asc');

  const handleSort = (key) => {
    if (sortKey === key) {
      setSortOrder(order => order === 'asc' ? 'desc' : 'asc');
    } else {
      setSortKey(key);
      setSortOrder('asc');
    }
  };

  const sortedData = useMemo(() => {
    if (!sortKey) return data;
    return [...data].sort((a, b) => {
      const valA = a[sortKey];
      const valB = b[sortKey];
      if (valA < valB) return sortOrder === 'asc' ? -1 : 1;
      if (valA > valB) return sortOrder === 'asc' ? 1 : -1;
      return 0;
    });
  }, [data, sortKey, sortOrder]);

  return (
    <table>
      <thead>
        <tr>
          {columns.map(col => (
            <th key={col.key} onClick={() => handleSort(col.key)} style={{ cursor: 'pointer' }}>
              {col.label} {sortKey === col.key ? (sortOrder === 'asc' ? '▲' : '▼') : ''}
            </th>
          ))}
        </tr>
      </thead>
      <tbody>
        {sortedData.map((row, i) => (
          <tr key={i}>
            {columns.map(col => (
              <td key={col.key}>{row[col.key]}</td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  );
}

```

```
    </table>

  );
}
```

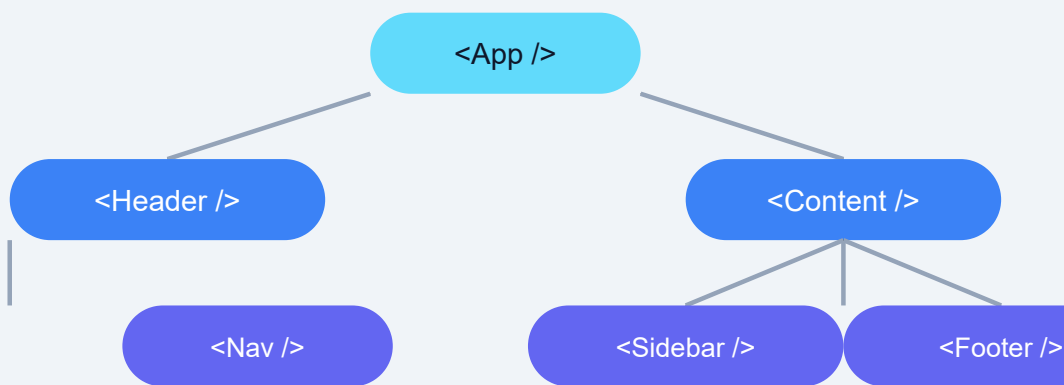
Integrating Maps with react-leaflet

React Leaflet provides React bindings for Leaflet, a leading open-source interactive map library. It lets you add maps, markers, and geo-layers to your React application using a declarative component API.

```
import { MapContainer, TileLayer, Marker, Popup } from 'react-leaflet';

function LocationMap({ lat, lng, name }) {
  return (
    <MapContainer center={[lat, lng]} zoom={13} style={{ height: '400px', width: '100%' }}>
      <TileLayer
        url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        attribution='&copy; OpenStreetMap contributors'
      />
      <Marker position={[lat, lng]}>
        <Popup>{name}</Popup>
      </Marker>
    </MapContainer>
  );
}
```

React Component Tree



React apps are built as a tree of reusable components.
Each component manages its own state and renders UI.

CSS Animations in React

CSS animations can be applied to React components using CSS classes and keyframes. You can toggle animation classes based on component state to create transitions and animated effects.

```
// CSS (animation.css)
// @keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
// .fade-in { animation: fadeIn 0.5s ease-in; }

function AnimatedMessage({ show, message }) {
  return (
    <div className={show ? 'fade-in' : ''} style={{ display: show ? 'block' :
      {message}
    </div>
  );
}
```

Framer Motion Basics

Framer Motion is a popular animation library for React. It provides a declarative API for creating animations, gestures, and layout transitions. The `motion` component extends standard HTML and SVG elements with animation capabilities.

```
import { motion, AnimatePresence } from 'framer-motion';

function AnimatedList({ items }) {
  return (
    <ul>
      <AnimatePresence>
        {items.map(item => (
          <motion.li key={item.id}
            initial={{ opacity: 0, x: -20 }}
            animate={{ opacity: 1, x: 0 }}
            exit={{ opacity: 0, x: 20 }}
            transition={{ duration: 0.3 }}
          >
            {item.text}
          </motion.li>
        ))}
      </AnimatePresence>
    </ul>
  );
}
```

Practice Task

Build a product catalog page with a grid layout using Material UI's `Grid` component. Display at least six product cards, each with an image placeholder, title, price, and an "Add to Cart" button. Use a sortable table view as a toggleable alternative to the grid. Add a Framer Motion animation when items appear or are removed from the view.

Forms in React

Controlled Components

In a controlled component, the form data is handled by the React component's state. The form element's value is always derived from state, and any changes are handled by an `onChange` event handler. This gives React complete control over the form's behavior and makes the form data predictable and easy to validate.

```
// Controlled input
function ControlledInput() {
  const [name, setName] = useState('');

  const handleChange = (e) => {
    setName(e.target.value);
  };

  return (
    <div>
      <input type="text" value={name} onChange={handleChange} />
      <p>You typed: {name}</p>
    </div>
  );
}
```

Textarea

```
function FeedbackForm() {
  const [feedback, setFeedback] = useState('');

  return (
    <div>
      <textarea value={feedback} onChange={e => setFeedback(e.target.value)}
        rows={5} cols={40} placeholder="Enter your feedback..." />
      <p>Characters: {feedback.length}</p>
    </div>
  );
}
```

Select

```
function CountrySelect() {
  const [country, setCountry] = useState('');

  return (
    <select value={country} onChange={e => setCountry(e.target.value)}>
      <option value="">Select a country</option>
      <option value="us">United States</option>
      <option value="uk">United Kingdom</option>
      <option value="ca">Canada</option>
      <option value="in">India</option>
    </select>
  );
}
```

Uncontrolled Components

Uncontrolled components store form data in the DOM itself rather than in React state. You access the form values using a `ref` (via `useRef`) when you need them, such as on form submission. Uncontrolled components can be useful for simple forms or when integrating with non-React code.

```
import { useRef } from 'react';

function UncontrolledForm() {
  const nameRef = useRef();
  const emailRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Name: ${nameRef.current.value}, Email: ${emailRef.current.value}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={nameRef} placeholder="Name" />
      <input ref={emailRef} type="email" placeholder="Email" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

When to Use Uncontrolled: Uncontrolled components are best for simple forms with few fields where you only need the values at submission time. Controlled components are preferred for complex forms with validation, conditional UI, or real-time feedback.

Formik Library

Formik is one of the most popular form management libraries for React. It handles form state, validation, error messages, and submission, reducing boilerplate significantly. Formik works well with Yup for schema-based validation.

```

import { useFormik } from 'formik';

function SignupForm() {
  const formik = useFormik({
    initialValues: {
      firstName: '',
      lastName: '',
      email: '',
    },
    onSubmit: (values) => {
      alert(JSON.stringify(values, null, 2));
    },
    validate: (values) => {
      const errors = {};
      if (!values.firstName) errors.firstName = 'Required';
      if (!values.lastName) errors.lastName = 'Required';
      if (!values.email) errors.email = 'Required';
      else if (!/^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$/i.test(values.email))
        errors.email = 'Invalid email address';
      }
      return errors;
    },
  });

  return (
    <form onSubmit={formik.handleSubmit}>
      <input name="firstName" value={formik.values.firstName} onChange={formik.handleChange} {formik.errors.firstName} && <div>{formik.errors.firstName}</div>
      <input name="lastName" value={formik.values.lastName} onChange={formik.handleChange} {formik.errors.lastName} && <div>{formik.errors.lastName}</div>
      <input name="email" type="email" value={formik.values.email} onChange={formik.handleChange} {formik.errors.email} && <div>{formik.errors.email}</div>
      <button type="submit">Submit</button>
    </form>
  );
}

```

Yup Validation Schema

Yup is a schema builder for value parsing and validation. When combined with Formik, it provides a clean, declarative way to define form validation rules.

```

import * as Yup from 'yup';
import { useFormik } from 'formik';

const validationSchema = Yup.object({
  username: Yup.string()
    .min(3, 'Must be at least 3 characters')
    .max(20, 'Must be 20 characters or less')
    .required('Required'),
  email: Yup.string()
    .email('Invalid email address')
    .required('Required'),
  age: Yup.number()
    .min(18, 'Must be at least 18')
    .max(120, 'Must be at most 120')
    .required('Required'),
  password: Yup.string()
    .min(8, 'Must be at least 8 characters')
    .matches(/[A-Z]/, 'Must contain an uppercase letter')
    .matches(/[0-9]/, 'Must contain a number')
    .required('Required'),
});

function RegistrationForm() {
  const formik = useFormik({
    initialValues: { username: '', email: '', age: '', password: '' },
    validationSchema,
    onSubmit: (values) => {
      console.log('Submitted:', values);
    },
  });

  return (
    <form onSubmit={formik.handleSubmit}>
      <input name="username" value={formik.values.username} onChange={formik.handleChange} {formik.errors.username} && <div>{formik.errors.username}</div> </input>
      <input name="email" value={formik.values.email} onChange={formik.handleChange} {formik.errors.email} && <div>{formik.errors.email}</div> </input>
      <input name="age" type="number" value={formik.values.age} onChange={formik.handleChange} {formik.errors.age} && <div>{formik.errors.age}</div> </input>
      <input name="password" type="password" value={formik.values.password} onChange={formik.handleChange} {formik.errors.password} && <div>{formik.errors.password}</div> </input>
      <button type="submit" disabled={formik.isSubmitting}>Register</button>
    </form>
  );
}

```

```
);  
}
```

Conditional Rendering

React offers several techniques for conditionally rendering elements. The three most common patterns are the ternary operator, the logical AND operator, and if/else statements outside the JSX.

```
function UserGreeting({ user }) {  
  // If/else outside JSX  
  if (!user) {  
    return <p>Please log in.</p>;  
  }  
  
  return (  
    <div>  
      <h1>Welcome, {user.name}!</h1>  
  
      { /* Ternary operator */}  
      <p>Role: {user.role === 'admin' ? 'Administrator' : 'User'}</p>  
  
      { /* Logical AND (&&) */}  
      {user.isVerified && <span>Verified Account</span>}  
  
      { /* Ternary for choosing between two options */}  
      {user.hasPremium ? (  
        <button>Access Premium Content</button>  
      ) : (  
        <button>Upgrade to Premium</button>  
      )}  
    </div>  
  );  
}
```

Lists and Keys

Rendering lists in React is done using the `map()` method to transform an array of data into an array of JSX elements. Each element in the rendered list must

have a unique `key` prop, which helps React identify which items have changed, been added, or been removed.

```
function FruitList() {
  const fruits = [
    { id: 1, name: 'Apple', color: 'red' },
    { id: 2, name: 'Banana', color: 'yellow' },
    { id: 3, name: 'Grape', color: 'purple' },
  ];

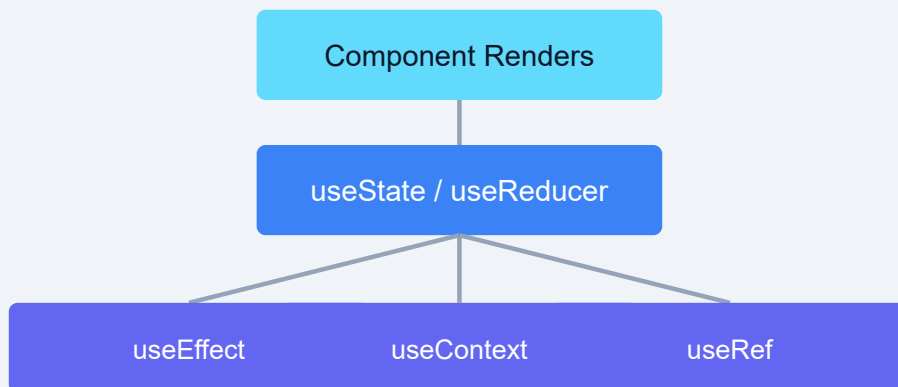
  return (
    <ul>
      {fruits.map(fruit => (
        <li key={fruit.id} style={{ color: fruit.color }}>
          {fruit.name}
        </li>
      ))}
    </ul>
  );
}

// Alternative: extracting to a component
function FruitItem({ fruit }) {
  return <li style={{ color: fruit.color }}>{fruit.name}</li>;
}

function FruitListRefactored() {
  const fruits = [
    { id: 1, name: 'Apple', color: 'red' },
    { id: 2, name: 'Banana', color: 'yellow' },
    { id: 3, name: 'Grape', color: 'purple' },
  ];

  return (
    <ul>
      {fruits.map(fruit => (
        <FruitItem key={fruit.id} fruit={fruit} />
      ))}
    </ul>
  );
}
```

React Hooks Data Flow



Hooks let you use state and lifecycle features in functional components.

Index as Key Anti-Pattern

Warning: Using the array index as a `key` is an anti-pattern. When items are reordered, added, or removed, React cannot correctly match elements across re-renders, leading to UI bugs and performance issues. Always use a stable, unique identifier (like a database ID) for keys.

```
// Anti-pattern (DO NOT USE)
{items.map((item, index) => (
  <li key={index}>{item.name}</li>
))}

// Correct approach
{items.map(item => (
  <li key={item.id}>{item.name}</li>
))}
```

Complete Form Example

```
function RegistrationForm() {
  const [form, setForm] = useState({
    username: '',
    email: '',
    password: '',
    acceptTerms: false,
  });
  const [errors, setErrors] = useState({});
  const [submitted, setSubmitted] = useState(false);

  const validate = () => {
    const errs = {};
    if (!form.username || form.username.length < 3) errs.username = 'Username';
    if (!form.email || !/\S+@\S+\.\S+/.test(form.email)) errs.email = 'Valid';
    if (!form.password || form.password.length < 6) errs.password = 'Password';
    if (!form.acceptTerms) errs.acceptTerms = 'You must accept the terms';
    return errs;
  };

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value,
    }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const validationErrors = validate();
    setErrors(validationErrors);

    if (Object.keys(validationErrors).length === 0) {
      setSubmitted(true);
      console.log('Form submitted:', form);
    }
  };

  if (submitted) {
    return <div>Thank you for registering!</div>;
  }
}
```

```

return (
  <form onSubmit={handleSubmit}>
    <div>
      <label>Username:</label>
      <input name="username" value={form.username} onChange={handleChange}
        {errors.username} && <span>{errors.username}</span>
      </div>
      <div>
        <label>Email:</label>
        <input name="email" type="email" value={form.email} onChange={handleChange}
          {errors.email} && <span>{errors.email}</span>
        </div>
        <div>
          <label>Password:</label>
          <input name="password" type="password" value={form.password} onChange={handleChange}
            {errors.password} && <span>{errors.password}</span>
          </div>
          <div>
            <label>
              <input name="acceptTerms" type="checkbox" checked={form.acceptTerms} />
              I accept the terms and conditions
            </label>
            {errors.acceptTerms} && <span>{errors.acceptTerms}</span>
          </div>
          <button type="submit">Register</button>
        </form>
      );
    }
  )

```

Practice Task

Build a `FeedbackSurvey` form with the following fields: full name (text input), rating (select dropdown from 1-5), comments (textarea), and subscribe to newsletter (checkbox). Use Formik for form management and Yup for validation. The name should be required, rating should be required and between 1-5, and comments should be optional with a max length of 500 characters. Display a success message after submission.

Routing & Navigation in React

Client-Side Routing

React is a single-page application (SPA) library. Without routing, you can only render one view at a time. Client-side routing lets you map URL paths to components without a full page reload. The dominant library is **react-router-dom** (v6+).

Setting Up React Router

Install with `npm install react-router-dom`. The core components are:

- `BrowserRouter` — wraps the app and listens to URL changes
- `Routes` / `Route` — declarative route definitions
- `Link` / `NavLink` — navigation elements

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "../pages/Home";
import About from "../pages/About";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

NavLink for Active States

`NavLink` adds a `className` or `style` callback when the link matches the current URL, making it easy to highlight the active page.

```
<NavLink
  to="/dashboard"
  className={({ isActive }) => (isActive ? "active" : "")}
>
  Dashboard
</NavLink>
```

Nested Routes

Routes can be nested to create layouts that share a common UI. Use `<Outlet />` to render the child route inside a parent layout.

```
import { Outlet, Link } from "react-router-dom";

function DashboardLayout() {
  return (
    <div>
      <h2>Dashboard</h2>
      <nav>
        <Link to="profile">Profile</Link>
        <Link to="settings">Settings</Link>
      </nav>
      <Outlet /> {/* child routes render here */}
    </div>
  );
}

// Route definition
<Route path="dashboard" element={<DashboardLayout />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

Nested routes help you avoid repeating layout code. The parent component owns the shared chrome while `Outlet` injects the child content.

Dynamic Routes & useParams

Use a colon (`:`) in the path to define a parameter segment. The `useParams` hook returns an object of matched parameters.

```
import { useParams } from "react-router-dom";

function UserProfile() {
  const { username } = useParams();
  return <h1>Profile of {username}</h1>;
}

// Route: <Route path="/user/:username" element={<UserProfile />} />
// URL: /user/snehal → username = "snehal"
```

Programmatic Navigation with useNavigate

Sometimes you need to navigate after an event (e.g. form submission). The `useNavigate` hook returns a navigation function.

```
import { useNavigate } from "react-router-dom";

function LoginPage() {
  const navigate = useNavigate();

  const handleLogin = async () => {
    await login();
    navigate("/dashboard", { replace: true });
  };

  return <button onClick={handleLogin}>Log In</button>;
}
```

404 Pages (Catch-All Route)

A route with `path="*"` matches any undefined path. Place it last inside `Routes` .

```
<Route path="*" element={<NotFound />} />
```

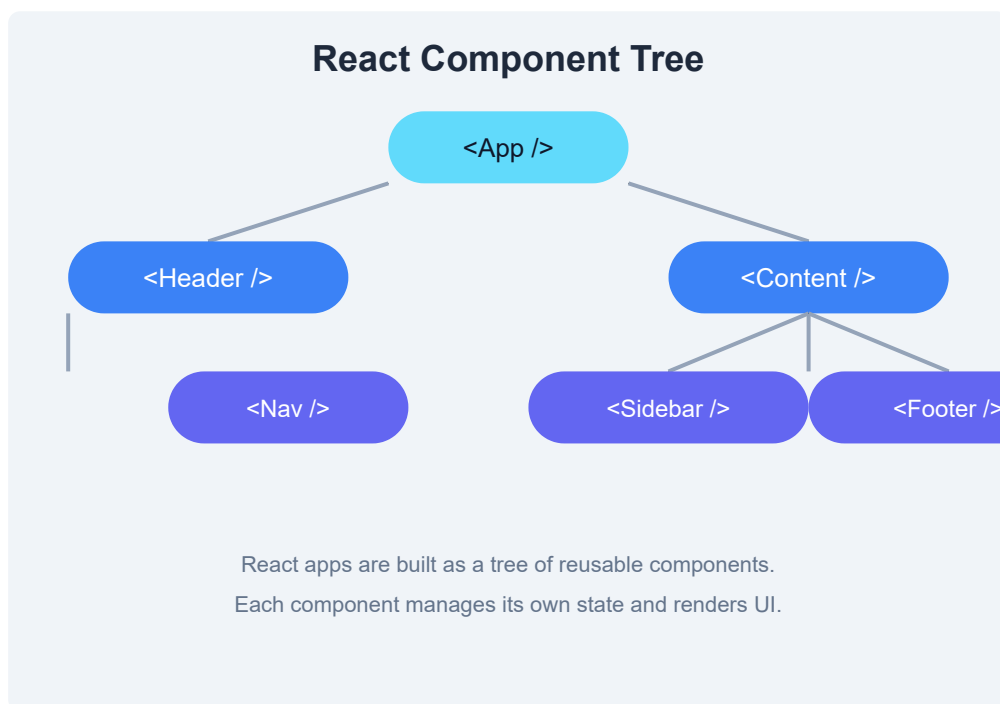
Protected Routes (Auth Guards)

Create a wrapper component that checks authentication and either renders the child route or redirects to login.

```
import { Navigate } from "react-router-dom";

function ProtectedRoute({ children }) {
  const isAuthenticated = useAuth(); // your auth hook
  return isAuthenticated ? children : <Navigate to="/login" replace />;
}

// Usage
<Route path="/admin" element={
  <ProtectedRoute><Admin /></ProtectedRoute>
} />
```



Route Query Parameters

Use the `useSearchParams` hook to read and update query string parameters (e.g. `?page=2&sort=name`).

```
import { useSearchParams } from "react-router-dom";

function ProductList() {
  const [searchParams, setSearchParams] = useSearchParams();
  const page = searchParams.get("page") || "1";

  const nextPage = () => {
    setSearchParams({ page: String(Number(page) + 1) });
  };

  return (
    <div>
      <h2>Products – Page {page}</h2>
      <button onClick={nextPage}>Next</button>
    </div>
  );
}
```

Lazy Loading Routes

Combine `React.lazy` and `Suspense` with React Router to code-split at the route level.

```
const Dashboard = React.lazy(() => import("../pages/Dashboard"));

<Routes>
  <Route path="/dashboard" element={
    <React.Suspense fallback={<div>Loading...</div>}>
      <Dashboard />
    </React.Suspense>
  } />
</Routes>
```

Lazy loading reduces the initial bundle size. Each route chunk is fetched only when the user navigates to that path.

Exercise: Build a Mini Wiki

1. Set up a new React app with `react-router-dom`.
2. Create routes for Home, About, and a dynamic `/article/:slug` page.
3. Add a `NavLink`-based navigation bar with active styling.
4. Create a `ProtectedRoute` that redirects to `/login` if a `user` state is null.
5. Add a 404 catch-all page.

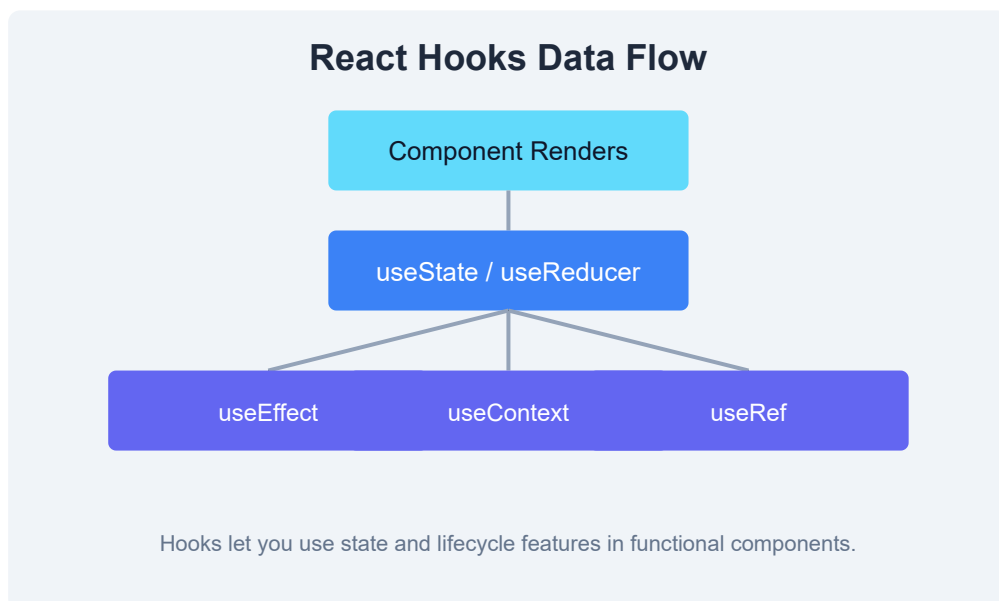
Challenge: Use `useSearchParams` to implement a search box that filters articles by title.

State Management in React

As your application grows, passing state through many levels of components (prop drilling) becomes cumbersome. State management libraries provide a central place to store and mutate shared state.

The Flux Pattern

Flux is an architectural pattern introduced by Facebook. Data flows in a single direction: **Action** → **Dispatcher** → **Store** → **View**. A user action (click, input) dispatches an action object to the store, which updates itself and notifies the view to re-render.



Context API

React's built-in `createContext` and `useContext` allow you to share values without prop drilling. It is ideal for global data like themes, locale, or current user.

```
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div>Current theme: {theme}</div>;
}
```

Context is not a state management tool by itself — it is a dependency injection mechanism. Combine it with `useReducer` to mimic Redux-like behavior without external libraries.

Redux

Redux is the most popular state management library. It follows the Flux pattern strictly: a single **store** holds the entire state tree, **actions** describe what happened, and **reducers** specify how the state changes.

```
// Action
const increment = () => ({ type: "counter/increment" });

// Reducer
function counterReducer(state = { count: 0 }, action) {
  switch (action.type) {
    case "counter/increment":
      return { count: state.count + 1 };
    default:
      return state;
  }
}

// Store
import { createStore } from "redux";
const store = createStore(counterReducer);

// Dispatch
store.dispatch(increment());
console.log(store.getState()); // { count: 1 }
```

React-Redux (connect)

The `connect` HOC maps state and dispatch to component props.

```
import { connect } from "react-redux";

function Counter({ count, increment }) {
  return <button onClick={increment}>{count}</button>;
}

const mapStateToProps = (state) => ({ count: state.count });
const mapDispatchToProps = { increment };

export default connect(mapStateToProps, mapDispatchToProps)(Counter);
```

Redux Toolkit (RTK)

Redux Toolkit is the modern, opinionated way to write Redux. It eliminates boilerplate with `configureStore` and `createSlice`.

```
import { configureStore, createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { count: 0 },
  reducers: {
    increment(state) { state.count += 1; },
    decrement(state) { state.count -= 1; },
  },
});

export const { increment, decrement } = counterSlice.actions;

const store = configureStore({
  reducer: { counter: counterSlice.reducer },
});
```

RTK includes `createAsyncThunk` for side effects, `createEntityAdapter` for normalized data, and DevTools out of the box. It is now the recommended way to write Redux.

Zustand

Zustand is a minimalist state management library with a hook-based API. No providers needed.

```
import { create } from "zustand";

const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
}));

function Counter() {
  const { count, increment } = useStore();
  return <button onClick={increment}>{count}</button>;
}
```

Recoil

Recoil is a state management library by Meta that uses **atoms** (units of state) and **selectors** (derived state). It integrates deeply with React's concurrent features.

```
import { atom, useRecoilState } from "recoil";

const countAtom = atom({
  key: "count",
  default: 0,
});

function Counter() {
  const [count, setCount] = useRecoilState(countAtom);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

When to Use Each

Tool	Best for
Context API + useReducer	Small apps, low-frequency updates, simple global state
Redux / Redux Toolkit	Large apps, complex state logic, mid-to-large teams
Zustand	Medium apps, minimal boilerplate, hook-native APIs
Recoil	Apps needing derived state and concurrent mode compatibility

Exercise: Build a Shopping Cart

1. Use Redux Toolkit to create a `cartSlice` with `addItem`, `removeItem`, and `clearCart` reducers.
2. Configure the store and wrap your app in `Provider`.
3. Create a `Cart` component that displays items and totals from the store.
4. Add a `ProductList` component that dispatches `addItem` on button click.
5. **Bonus:** Re-implement the same app using Zustand and compare the code size.

Advanced React Concepts

Accessibility (a11y)

React fully supports HTML ARIA attributes. Use `aria-*` attributes on JSX elements just like in HTML. Manage focus with `useRef` and the `tabIndex` prop.

```
function Modal({ isOpen, onClose }) {
  const closeRef = useRef(null);

  useEffect(() => {
    if (isOpen) closeRef.current?.focus();
  }, [isOpen]);

  return isOpen ? (
    <div role="dialog" aria-modal="true" aria-labelledby="modal-title">
      <h2 id="modal-title">Confirm</h2>
      <button ref={closeRef} onClick={onClose}>Close</button>
    </div>
  ) : null;
}
```

Code Splitting with React.lazy & Suspense

`React.lazy` lets you dynamically import components. Wrap them in `Suspense` with a fallback UI while the chunk loads.

```
const HeavyChart = React.lazy(() => import("./HeavyChart"));

function Dashboard() {
  return (
    <React.Suspense fallback={<div>Loading chart...</div>}>
      <HeavyChart />
    </React.Suspense>
  );
}
```

Error Boundaries

Error boundaries catch JavaScript errors in their child tree and display a fallback UI instead of crashing the whole app. They must be class components implementing `componentDidCatch` and/or `getDerivedStateFromError`.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() { return { hasError: true }; }

  componentDidCatch(error, info) {
    logErrorToService(error, info.componentStack);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}
```

forwardRef

`forwardRef` lets a parent component pass a `ref` down to a child's DOM element.

```
const FancyInput = React.forwardRef((props, ref) => (  
  <input ref={ref} className="fancy" {...props} />  
));  
  
// Parent  
const ref = useRef();  
<FancyInput ref={ref} />  
ref.current.focus();
```

Fragments

`<>...</>` or `<React.Fragment>` lets you group children without adding extra nodes to the DOM.

```
function List() {  
  return (  
    <>  
      <dt>Term</dt>  
      <dd>Definition</dd>  
    </>  
  );  
}
```

Higher-Order Components (HOCs)

An HOC is a function that takes a component and returns a new component with added props or logic.

```
function withLogger(WrappedComponent) {
  return function Enhanced(props) {
    console.log("Rendering", WrappedComponent.name);
    return <WrappedComponent {...props} />;
  };
}

const EnhancedButton = withLogger(MyButton);
```

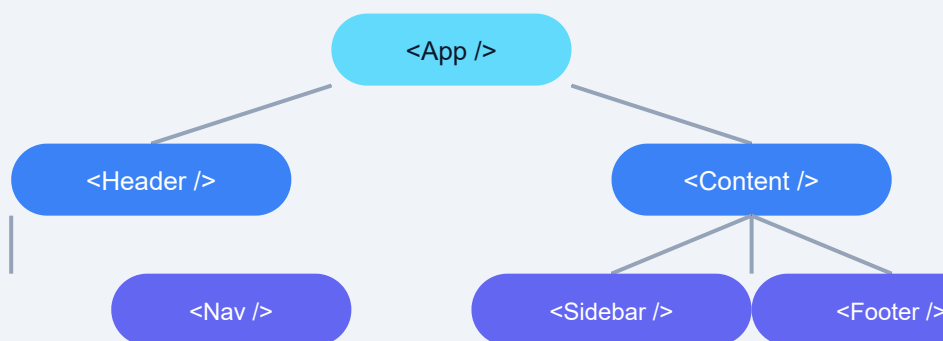
Render Props

A render prop is a function prop a component uses to render its content, sharing internal state.

```
function MouseTracker({ render }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return <div onMouseMove={(e) => setPos({ x: e.clientX, y: e.clientY })}>
    {render(pos)}
  </div>;
}

// Usage
<MouseTracker render={({ x, y }) => <h1>{x}, {y}</h1>} />
```

React Component Tree



React apps are built as a tree of reusable components.
Each component manages its own state and renders UI.

Performance: React.memo, useMemo, useCallback

- `React.memo` — prevents re-render when props haven't changed (shallow comparison).
- `useMemo` — memoizes the result of an expensive calculation.
- `useCallback` — memoizes a function reference so child components don't re-render unnecessarily.

```
const ExpensiveList = React.memo(({ items }) => (  
  <ul>{items.map((i) => <li key={i.id}>{i.name}</li>)}</ul>  
));  
  
function App() {  
  const [count, setCount] = useState(0);  
  
  const sorted = useMemo(() => items.sort((a, b) => a.name.localeCompare(b.name)), [items]);  
  const onRemove = useCallback((id) => removeItem(id), []);  
  
  return <>  
    <button onClick={() => setCount(c => c + 1)}>{count}</button>  
    <ExpensiveList items={sorted} onRemove={onRemove} />  
  </>;  
}
```

Portals

`createPortal` renders children into a different DOM node, useful for modals, tooltips, and dropdowns.

```
import { createPortal } from "react-dom";  
  
function Modal({ children }) {  
  return createPortal(  
    <div className="modal-overlay">{children}</div>,  
    document.getElementById("portal-root")  
  );  
}
```

Profiler API

The `Profiler` component measures rendering performance of its subtree.

```
<Profiler
  id="Navigation"
  onRender={({id, phase, actualDuration}) => {
    console.log({ id, phase, actualDuration });
  }}
>
  <Navigation />
</Profiler>
```

Reconciliation & Strict Mode

React's reconciliation algorithm compares the virtual DOM tree and applies minimal DOM updates. Keys (`key` prop) help identify which items changed. `StrictMode` double-invokes effects in development to surface side-effect bugs.

Exercise: Add Error Handling & Performance Tracking

1. Create an `ErrorBoundary` class component that shows a retry button on error.
2. Use `React.lazy` to lazy-load a heavy chart component.
3. Wrap a navigation component in a `Profiler` and log render durations.
4. Create a `withAuth` HOC that passes a `user` prop.
5. Use `createPortal` to render a modal outside the root DOM hierarchy.

Testing & Deployment

Why Test React Apps?

Automated tests catch regressions, document expected behavior, and give you confidence when refactoring. The React ecosystem standard on **Jest** (test runner) and **React Testing Library** (RTL) for component tests.

Unit Testing with Jest & RTL

Jest provides `describe`, `it / test`, `expect`, and `mock`. RTL provides utilities to render components and query rendered output.

```
import { render, screen, fireEvent } from "@testing-library/react";
import Counter from "../Counter";

test("increments count when button is clicked", () => {
  render(<Counter />);
  const button = screen.getByRole("button", { name: /increment/i });
  fireEvent.click(button);
  expect(screen.getByText("1")).toBeInTheDocument();
});
```

Snapshot Testing

Snapshot tests capture the rendered output and compare it to a stored reference file. Use them for UI components that rarely change.

```
import { render } from "@testing-library/react";
import Header from "../Header";

test("Header matches snapshot", () => {
  const { container } = render(<Header title="Home" />);
  expect(container.firstChild).toMatchSnapshot();
});
```

Snapshots are stored in `__snapshots__` folders alongside your test file. Run `jest --updateSnapshot` to refresh them after intentional changes.

Testing Hooks

Use `renderHook` from RTL to test custom hooks outside of a component.

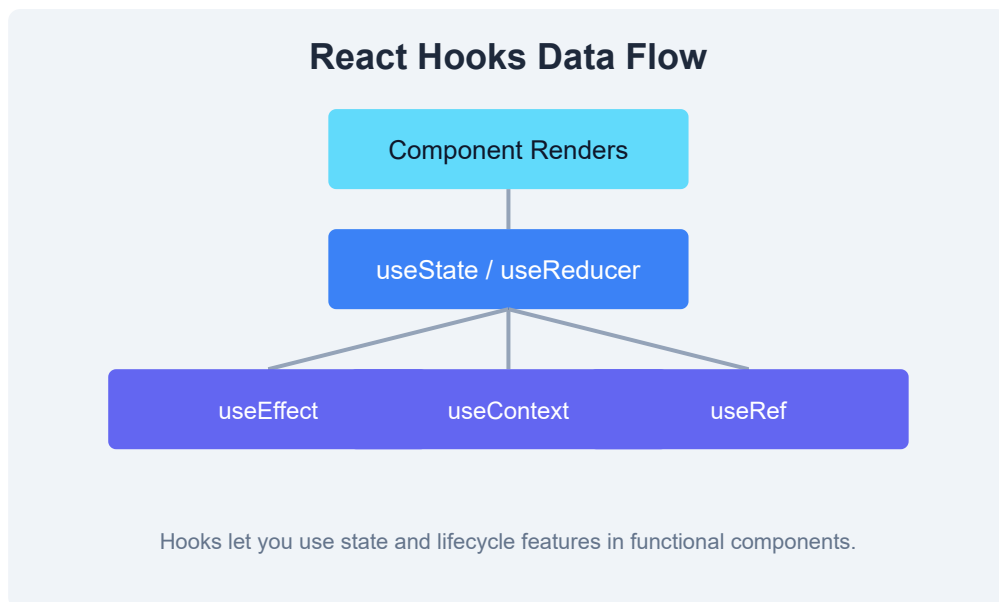
```
import { renderHook, act } from "@testing-library/react";
import useCounter from "../useCounter";

test("useCounter increments value", () => {
  const { result } = renderHook(() => useCounter(0));
  act(() => result.current.increment());
  expect(result.current.count).toBe(1);
});
```

React CLI Commands

Create React App provides four commands via `react-scripts`:

- `npm start` — starts dev server on `localhost:3000`
- `npm run build` — creates an optimized production build in `build/`
- `npm test` — runs Jest in watch mode
- `npm run eject` — exposes all config files (irreversible)



Building for Production

Running `npm run build` generates minified, tree-shaken bundles with content hashes for cache busting. The `build/` folder can be served by any static file server.

```
npx create-react-app my-app
cd my-app
npm run build
# Output in build/ ready for deployment
```

Deploying to Vercel

Vercel is optimized for frontend apps. Connect your Git repository or use the CLI:

```
npm install -g vercel
vercel
# Follow prompts — it auto-detects Create React App
```

Deploying to Netlify

Netlify supports drag-and-drop of the `build/` folder or Git-based CI/CD. Set the build command to `npm run build` and publish directory to `build`.

Deploying to GitHub Pages

For GitHub Pages, install `gh-pages` and add a `homepage` field to `package.json`.

```
npm install --save-dev gh-pages

// package.json
"homepage": "https://yourname.github.io/my-app",
"scripts": {
  "predeploy": "npm run build",
  "deploy": "gh-pages -d build"
}

npm run deploy
```

Example Project Walkthrough

A typical full-stack React project might include a Node/Express API, a PostgreSQL database, and a React frontend. Alternatively, a static site (e.g. a portfolio blog) is deployed directly from the `build/` folder to any static host.

For SPAs with client-side routing, ensure your host redirects all paths to `index.html` (Vercel and Netlify do this automatically; for GitHub Pages you need a `404.html` or a custom `.htaccess`).

Exercise: Test and Deploy a React App

1. Write a unit test for a `TodoList` component using RTL — test render, add, and delete.
2. Add a snapshot test for a `Header` component.
3. Run `npm run build` and inspect the output in the `build/` folder.
4. Deploy the built app to Vercel or Netlify using either the CLI or Git-based workflow.
5. Configure redirect rules so that client-side routes work on refresh.

Additional Features & Libraries

Beyond the core React API, the ecosystem offers many utility libraries that solve common problems. This chapter covers popular third-party packages and React's lower-level APIs.

react-datepicker

A lightweight calendar date picker component with excellent i18n support.

```
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";

function MyDatePicker() {
  const [startDate, setStartDate] = useState(new Date());
  return (
    <DatePicker
      selected={startDate}
      onChange={(date) => setStartDate(date)}
      dateFormat="yyyy/MM/dd"
      minDate={new Date()}
    />
  );
}
```

react-helmet-async

Manage the document head (title, meta tags, links) from your React components. This is essential for SEO and social sharing.

```
import { Helmet } from "react-helmet-async";

function BlogPost({ title, description }) {
  return (
    <>
      <Helmet>
        <title>{title} – My Blog</title>
        <meta name="description" content={description} />
        <meta property="og:title" content={title} />
      </Helmet>
      <article>...</article>
    </>
  );
}
```

Inline Styles

React supports a `style` prop that accepts a JavaScript object with camelCased CSS properties. It is useful for dynamic styles, but avoid it for static styling — use CSS classes instead.

```
function DynamicBox({ color }) {
  return (
    <div style={{
      backgroundColor: color,
      padding: "16px",
      borderRadius: "8px",
      width: color ? "200px" : "100px",
    }}>
      Hello
    </div>
  );
}
```

PropTypes

`PropTypes` provides runtime type-checking for props. While TypeScript has largely replaced it, PropTypes is still useful for JavaScript-only projects.

```
import PropTypes from "prop-types";

function UserCard({ name, age, isActive }) {
  return (<div>{name} ({age}) – {isActive ? "Active" : "Inactive"}</div>);
}

UserCard.propTypes = {
  name: PropTypes.string.isRequired,
  age: PropTypes.number,
  isActive: PropTypes.bool,
};

UserCard.defaultProps = {
  age: 0,
  isActive: false,
};
```

PropTypes warnings only appear in development mode. In production builds they are stripped out for performance.

react-responsive-carousel

A flexible image carousel/slider component with swipe support, autoplay, and custom rendering.

```
import { Carousel } from "react-responsive-carousel";
import "react-responsive-carousel/lib/styles/carousel.min.css";

function Gallery({ images }) {
  return (
    <Carousel showThumbs={false} autoPlay infiniteLoop>
      {images.map((src, i) => (
        <div key={i}>
          <img src={src} alt={`Slide ${i + 1}`} />
        </div>
      ))}
    </Carousel>
  );
}
```

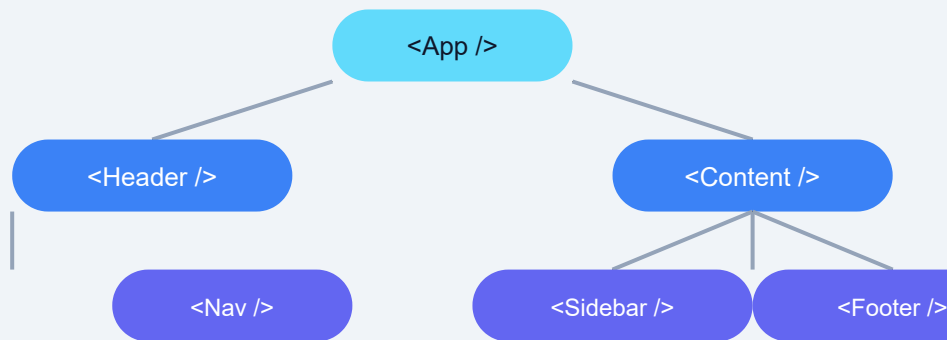
react-icons

A comprehensive icon library that imports popular icon sets (Font Awesome, Material Icons, Feather, etc.) as React components. Only the icons you use end up in your bundle.

```
import { FaReact } from "react-icons/fa";
import { MdHome } from "react-icons/md";
import { FiSun } from "react-icons/fi";

function IconsDemo() {
  return (
    <div>
      <FaReact size={32} color="#61DAFB" />
      <MdHome size={28} />
      <FiSun className="sun-icon" />
    </div>
  );
}
```

React Component Tree



React apps are built as a tree of reusable components.
Each component manages its own state and renders UI.

React Reference API

React provides low-level utilities that power many libraries and advanced patterns:

- `React.createElement(type, props, ...children)` — creates a React element (what JSX compiles to).
- `React.cloneElement(element, props, ...children)` — clones an element with merged new props.
- `React.Children` — utilities for manipulating `props.children` (map, forEach, count, only, toArray).
- `React.isValidElement` — checks if a value is a valid React element.

```
// createElement example (equivalent to JSX)
const element = React.createElement("h1", { className: "title" }, "Hello");

// cloneElement example
const cloned = React.cloneElement(child, { expanded: true });

// Children utilities
React.Children.map(children, (child) => (
  <li>{child}</li>
));
```

Exercise: Build a Dashboard with Custom Features

1. Use `react-datepicker` to add a date range filter to a dashboard.
2. Use `react-helmet-async` to set per-page titles and meta descriptions.
3. Add a responsive image gallery using `react-responsive-carousel`.
4. Use `react-icons` to add social media icons to a footer.
5. Define `PropTypes` for all your components.
6. **Bonus:** Use `React.cloneElement` to create a `Focusable` wrapper that auto-focuses its child on mount.

Practice & Resources

This chapter consolidates everything from the tutorial into a quick reference guide, a cheatsheet, study resources, and interview preparation material. Bookmark this page for daily practice.

React Quick Reference

Pattern	Code
Functional component	<code>const Comp = ({ name }) => <h1>{name}</h1>;</code>
useState	<code>const [count, setCount] = useState(0);</code>
useEffect	<code>useEffect(() => { ... }, [deps]);</code>
useRef	<code>const ref = useRef(null); ref.current.focus();</code>
Conditional render	<code>{isLoggedIn ? <Dash /> : <Login />}</code>
List rendering	<code>items.map(i => <Item key={i.id} />)</code>
Event handling	<code><button onClick={() => handle()}>Click</button></code>

Hooks Summary

- `useState` — local component state
- `useEffect` — side effects (data fetching, subscriptions, DOM manipulation)
- `useContext` — consume a React context
- `useReducer` — complex state logic with reducer functions
- `useCallback` — memoize a callback function
- `useMemo` — memoize a computed value
- `useRef` — mutable ref to DOM element or value
- `useImperativeHandle` — customize exposed ref methods
- `useLayoutEffect` — synchronous effect after DOM mutations
- `useDebugValue` — label custom hooks in DevTools
- `useTransition` — mark state update as non-urgent

- `useDeferredValue` — defer re-rendering of a value
- `useId` — generate unique IDs for accessibility
- `useSyncExternalStore` — subscribe to external stores

Axios Cheatsheet

```
import axios from "axios";

// GET
const { data } = await axios.get("/api/users");

// POST
await axios.post("/api/users", { name, email });

// PUT
await axios.put("/api/users/1", { name });

// DELETE
await axios.delete("/api/users/1");

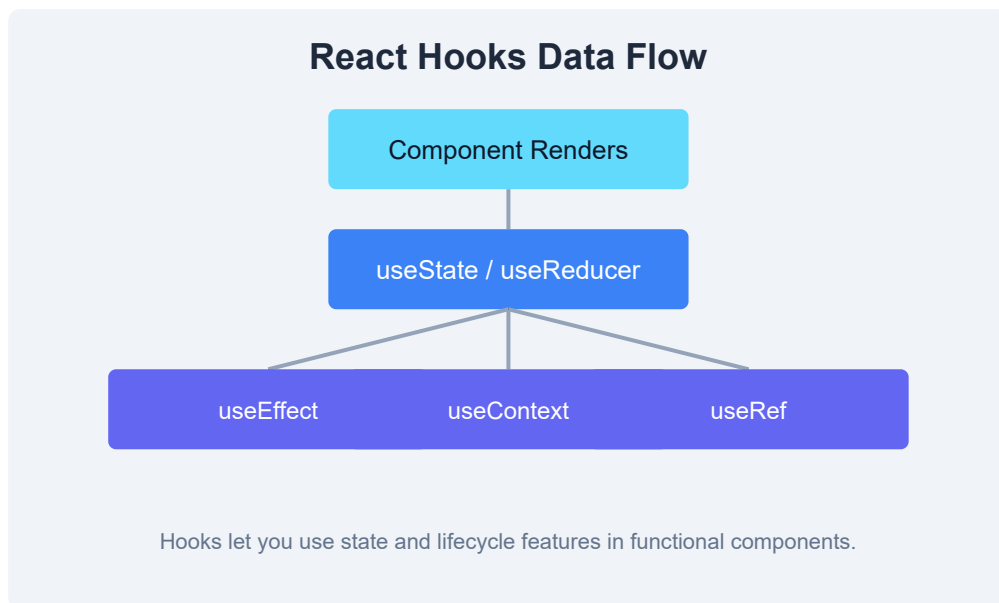
// Interceptors
axios.interceptors.request.use((config) => {
  config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Error handling
try {
  const { data } = await axios.get("/api/data");
} catch (err) {
  if (err.response?.status === 404) { /* not found */ }
}
```

Useful Resources

- **React Docs** — react.dev (official docs with interactive examples)
- **Awesome React** — curated list of React tools and resources on GitHub
- **React TypeScript Cheatsheet** — patterns for typing React components with TypeScript

- **React DevTools** — browser extension for profiling and inspecting component trees
- **Discussion Forums:** Reactiflux Discord, Stack Overflow, Reddit r/reactjs



8-Week Study Plan

1. **Week 1-2:** JSX, components, props, useState, conditional rendering, lists
2. useEffect, data fetching, custom hooks
3. Forms, refs, context, useReducer
4. React Router, navigation patterns
5. State management (Redux Toolkit or Zustand)
6. Testing, performance, accessibility
7. Build a capstone project + mock interviews

Interview Prep Questions

- What is the virtual DOM and how does React use it?
- Explain the difference between controlled and uncontrolled components.
- How does the useEffect dependency array work?
- What is prop drilling and how can you avoid it?
- Explain the Flux pattern and how Redux implements it.
- What is the purpose of keys in React lists?
- How do you optimize performance in a React app?
- What are error boundaries and what are their limitations?

- Explain the difference between `useMemo` and `useCallback`.
- How does `React.lazy` help with code splitting?

Glossary of Terms

Component

A reusable, self-contained piece of UI, written as a function or class.

JSX

Syntax extension for JavaScript that looks like HTML, transpiled to

`React.createElement` calls.

Virtual DOM

An in-memory representation of the real DOM. React diffs it against the updated version to apply minimal DOM patches.

Hook

Functions that let you use React features (state, effects, refs) inside functional components.

Props

Read-only data passed from parent to child components.

State

Data that changes over time and triggers re-renders when updated.

Exercise: Interview Simulation

1. Write answers to all 10 interview questions above.
2. Build a small app (e.g. a GitHub user search) using the full stack of tools learned in the tutorial.
3. Share your capstone project on GitHub and deploy it to Vercel or Netlify.
4. Review your portfolio and write a brief case study for each project.

React Reference

This chapter serves as a comprehensive API reference for React 18+. Use it as a quick lookup when you need the exact signature or behavior of a built-in API.

Top-Level React API

These are exported from the `react` package and available at the top level:

React.createElement

```
React.createElement(type, props, ...children)
// Creates a React element. JSX compiles to this.
// type: tag name string | React component | React.Fragment
// props: object or null
// children: any number of child nodes

React.createElement("div", { className: "box" }, "Hello");
```

React.createRef

```
React.createRef()
// Creates a ref object: { current: null }
// Used in class components. In function components, use useRef instead.

class MyComp extends React.Component {
  inputRef = React.createRef();
  focus = () => this.inputRef.current.focus();
  render() { return <input ref={this.inputRef} />; }
}
```

React.forwardRef

```
React.forwardRef((props, ref) => ReactElement)
// Lets parent components pass a ref through to a child DOM element.

const FancyButton = React.forwardRef(({ children }, ref) => (
  <button ref={ref} className="fancy">{children}</button>
));
```

React.lazy

```
React.lazy(() => import("./Component"))
// Returns a dynamically loaded component. Must be wrapped in Suspense.

const LazyChart = React.lazy(() => import("./Chart"));
```

React.memo

```
React.memo(Component, [areEqual])
// Memoizes a component – only re-renders if props changed (shallow compare).
// Optional areEqual(prevProps, nextProps) for custom comparison.

const Purelist = React.memo(({ items }) => (
  <ul>{items.map(i => <li key={i}>{i}</li>)}</ul>
));
```

React.startTransition

```
React.startTransition(callback)
// Marks a state update as a transition (non-urgent).
// Urgent updates (input, clicks) are not blocked by transitions.

startTransition(() => {
  setSearchQuery(query);
});
```

DOM Reference

These are exported from `react-dom` and interact with the browser DOM.

createRoot

```
import { createRoot } from "react-dom/client";
const root = createRoot(container);
root.render(<App />);
// Replaces ReactDOM.render in React 18+. Creates a concurrent root.
```

hydrateRoot

```
import { hydrateRoot } from "react-dom/client";
hydrateRoot(container, <App />);
// Hydrates server-rendered HTML. Used with frameworks like Next.js.
```

render (Legacy)

```
import ReactDOM from "react-dom";
ReactDOM.render(<App />, container);
// Legacy API – use createRoot for new projects.
```

findDOMNode (Legacy)

```
ReactDOM.findDOMNode(componentInstance);
// Returns the underlying DOM node. Avoid – use refs instead.
```

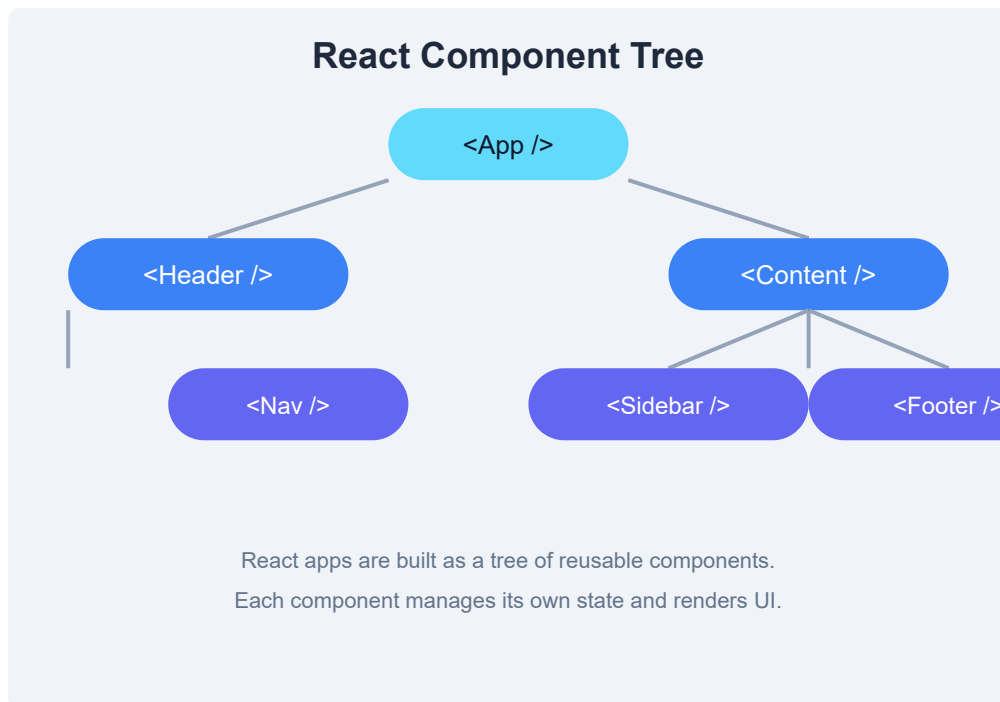
Hooks API Reference

All built-in hooks with their type signatures:

Hook	Signature	Description
useState	<code>useState<S>(initial: S (() => S)): [S, Dispatch<SetStateAction<S>>]</code>	Local state with lazy initializer
useEffect	<code>useEffect(effect: () => (void (() => void)), deps?: any[]): void</code>	Side effects after paint; cleanup runs on unmount/deps change
useLayoutEffect	<code>useLayoutEffect(effect, deps?): void</code>	Same as <code>useEffect</code> but fires synchronously after DOM mutations
useContext	<code>useContext<T>(ctx: Context<T>): T</code>	Reads current context value
useReducer	<code>useReducer<S, A>(reducer: (S, A) => S, initial: S, init?: (S) => S): [S, Dispatch<A>]</code>	Complex state with reducer pattern
useCallback	<code>useCallback<T extends Fn>(fn: T, deps: any[]): T</code>	Memoizes a function reference
useMemo	<code>useMemo<T>(factory: () => T, deps: any[]): T</code>	Memoizes a computed value
useRef	<code>useRef<T>(initial: T): MutableRefObject<T></code>	Mutable ref that persists across renders
useImperativeHandle	<code>useImperativeHandle(ref, createHandle, deps?): void</code>	Customizes ref exposed from <code>forwardRef</code>
useDebugValue	<code>useDebugValue(value, format?): void</code>	Labels custom hooks in React DevTools
useTransition	<code>useTransition(): [isPending: boolean, startTransition: (callback) => void]</code>	Marks state update as non-urgent
useDeferredValue	<code>useDeferredValue<T>(value: T): T</code>	Returns a deferred version of a value
useId	<code>useId(): string</code>	Generates unique IDs for accessibility attributes

`useSyncExternalStore` `useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?): T`

Subscribe to external non-React stores



SyntheticEvent Reference

React wraps native events in a `SyntheticEvent` for cross-browser consistency.
Common event types:

- `onClick` , `onDoubleClick`
- `onChange` , `onInput` , `onSubmit`
- `onKeyDown` , `onKeyUp` , `onKeyPress`
- `onFocus` , `onBlur`
- `onMouseEnter` , `onMouseLeave` , `onMouseMove`
- `onTouchStart` , `onTouchEnd` , `onTouchMove`
- `onScroll` , `onWheel`
- `onDragStart` , `onDragOver` , `onDrop`

```
function handleEvent(e: SyntheticEvent) {
  e.preventDefault();
  e.stopPropagation();
  console.log(e.type);      // "click"
  console.log(e.target);    // native DOM element
  console.log(e.currentTarget); // element the handler is attached to
  console.log(e.nativeEvent); // raw native event
}
```

Common Recipes

- **Fetch on mount:** `useEffect(() => { fetch(url).then(setData) }, [])`
- **Debounced input:** combine `useState` + `useEffect` with `setTimeout`
- **Previous state:** `const prev = useRef(value); useEffect(() => { prev.current = value; });`
- **Toggle:** `const [on, setOn] = useState(false); <button onClick={() => setOn(!on)} />`
- **Form state:** `const [form, setForm] = useState({}); setForm(prev => ({ ...prev, [name]: value }));`
- **Interval:** `useEffect(() => { const id = setInterval(tick, 1000); return () => clearInterval(id); }, [])`
- **Click outside:** attach a ref and check `!ref.current?.contains(e.target)`

This reference covers React 18 stable APIs. For experimental APIs, refer to the official RFCs and the React documentation at react.dev/reference/react.

Exercise: Build a Recipe Book Portal

1. Use `createPortal` to render a details panel overlay.
2. Implement `forwardRef` on an `Input` component so parents can call `focus()`.
3. Create an `ErrorBoundary` class component to catch rendering errors.
4. Use `React.memo` and `useCallback` to optimize a list of recipe cards.
5. Use `useDeferredValue` to keep the search input responsive while filtering a large list.